Lập trình mạng · Chương 1

Stream: dữ liệu có kiểu với DataStream và ObjectStream

TS. Đặng Ngọc Hùng

Hai đầu phải thống nhất cách đổi giá trị có kiểu thành byte

1. Byte stream chỉ truyền **byte**; chương trình lại cần gửi số nguyên, số thực, chuỗi, đối tượng
2. Bên gửi đổi giá trị thành byte theo một cách, bên nhận ghép theo cách khác: **giá trị sai**

> **Câu hỏi của phần:** hai chương trình thống nhất cách đổi giá trị có kiểu thành byte, và ngược lại, như thế nào?

writeInt ghi một số int thành đúng 4 byte, byte cao trước



Mỗi kiểu một định dạng cố định

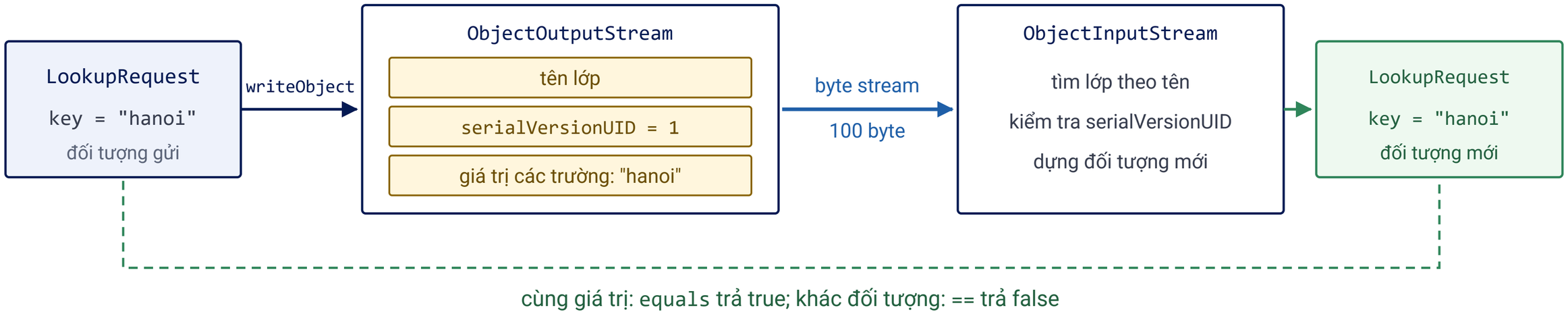
`writeInt: 4 byte` `writeLong: 8 byte` `writeDouble: 8 byte`

`writeUTF("hanoi"): 2 byte độ dài (00 05) + 5 byte`

Kiểu nguyên thủy có **kích thước cố định**; DataStream ghi theo thứ tự **big-endian**: byte cao trước.

Hai đầu phải đọc **đúng thứ tự, đúng kiểu** như khi ghi.

Serialization ghi đối tượng thành byte, bên nhận dựng một đối tượng mới



Serialization: đối tượng → byte (tên lớp, serialVersionUID, giá trị trường); bên nhận dựng **đối tượng mới**.

Hai đầu cần **cùng lớp** và cùng serialVersionUID.

DataStream ghi int, double và chuỗi thành 19 byte theo định dạng cố định

Java

```
var bytes = new ByteArrayOutputStream();  // ở vị trí stream của socket
try (var out = new DataOutputStream(bytes)) {
    out.writeInt(9199);
    out.writeDouble(1.5);
    out.writeUTF("hanoi");
}
System.out.println(HexFormat.ofDelimiter(" ").withUpperCase().formatHex(bytes.toByteArray()));
try (var in = new DataInputStream(new ByteArrayInputStream(bytes.toByteArray()))) {
    System.out.println(in.readInt() + " " + in.readDouble() + " " + in.readUTF());
}
```

Kết quả chạy

```
00 00 23 EF 3F F8 00 00 00 00 00 00 00 05 68 61 6E 6F 69
9199 1.5 hanoi
```

- `00 00 23 EF`: số 9199 · 8 byte: số 1.5 · `00 05`: độ dài chuỗi · 5 byte: `hanoi`

Đối tượng đọc lại có cùng giá trị nhưng không phải đối tượng đã gửi

Java

```
var sent = new LookupRequest("hanoi");
var bytes = new ByteArrayOutputStream();  // ở vị trí stream của socket
try (var out = new ObjectOutputStream(bytes)) {
    out.writeObject(sent);
}
System.out.println("serialized: " + bytes.size() + " bytes");
try (var in = new ObjectInputStream(new ByteArrayInputStream(bytes.toByteArray()))) {
    var received = (LookupRequest) in.readObject();
    System.out.println("equals: " + sent.equals(received) + ", same object: " + (sent == received));
}
```

Kết quả chạy

```
serialized: 100 bytes
equals: true, same object: false
```

- **Cùng giá trị** (`equals` trả true) nhưng là **đối tượng khác** (`==` trả false)
- **100 byte** cho một chuỗi 5 ký tự: phần lớn là tên lớp và mô tả lớp

Đọc sai thứ tự không báo lỗi mà cho giá trị sai

```java
var bytes = new ByteArrayOutputStream();
try (var out = new DataOutputStream(bytes)) {
    out.writeInt(9199);
    out.writeDouble(1.5);
}
try (var in = new DataInputStream(new ByteArrayInputStream(bytes.toByteArray()))) {
    System.out.println(in.readDouble());  // SAI: phía ghi đã ghi int trước
    System.out.println(in.readInt());
}
```

Kết quả chạy

```
1.95207695210046E-310
0
```

- Sai thứ tự, sai kiểu: còn đủ byte thì **không có ngoại lệ**, chỉ có giá trị sai; thiếu byte thì `EOFException`
- Serialization: khác serialVersionUID thì `InvalidClassException`; đọc đối tượng từ **nguồn không tin cậy** là lỗ hổng bảo mật
- Giao thức mạng nên dùng **định dạng tường minh**: dòng văn bản, JSON, Protocol Buffers

Câu hỏi 1

writeInt(9199) ghi bao nhiêu byte, theo thứ tự nào?

A. 2 byte: 23 EF.

B. 4 byte: EF 23 00 00, byte thấp trước.

C. 4 byte: 00 00 23 EF, byte cao trước.

D. 4 ký tự '9', '1', '9', '9' theo mã ASCII.

Câu hỏi 1

writeInt(9199) ghi bao nhiêu byte, theo thứ tự nào?

A 2 byte: 23 EF.

B 4 byte: EF 23 00 00, byte thấp trước.

C 4 byte: 00 00 23 EF, byte cao trước.

D 4 ký tự '9', '1', '9', '9' theo mã ASCII.

int luôn chiếm 4 byte; DataOutputStream ghi theo big-endian, byte cao trước: 00 00 23 EF. Ghi số thành các ký tự chữ số là cách của giao thức văn bản, không phải writeInt.

Câu hỏi 2

Đối tượng đọc từ ObjectInputStream có phải chính đối tượng đã gửi không?

A. Phải, vì Java truyền tham chiếu tới đối tượng qua mạng.

B. Không: đó là đối tượng mới có cùng giá trị; equals có thể trả true nhưng == trả false.

C. Phải, nếu hai chương trình chạy trên cùng một máy.

D. Không, và các trường của đối tượng mới luôn mang giá trị mặc định.

Câu hỏi 2

Đối tượng đọc từ ObjectInputStream có phải chính đối tượng đã gửi không?

A. Phải, vì Java truyền tham chiếu tới đối tượng qua mạng.

B. Không: đó là đối tượng mới có cùng giá trị; equals có thể trả true nhưng == trả false.

C. Phải, nếu hai chương trình chạy trên cùng một máy.

D. Không, và các trường của đối tượng mới luôn mang giá trị mặc định.

Serialization truyền tên lớp và giá trị các trường; bên nhận dựng một đối tượng mới và gán lại giá trị. Tham chiếu không đi qua mạng, kể cả khi hai chương trình cùng một máy.

Tổng kết

1. **DataStream**: kiểu nguyên thủy có kích thước cố định, ghi theo **big-endian**; đọc đúng thứ tự, đúng kiểu

2. **Serialization**: ghi tên lớp, serialVersionUID, giá trị trường; bên nhận dựng **đối tượng mới**

3. Đọc sai thứ tự không báo lỗi; serialization không an toàn với dữ liệu không tin cậy → giao thức mạng dùng định dạng tường minh

Từ khóa cần nhớ DataOutputStream · DataInputStream · kiểu dữ liệu nguyên thủy · thứ tự byte · big-endian · serialization · Serializable · serialVersionUID

Mở rộng

Chủ đề nâng cao

- ObjectInputFilter: giới hạn lớp được phép deserialize
- So sánh kích thước: serialization của Java, JSON, Protocol Buffers (Chương 4)

Bài tập mở rộng

Thiết kế message nhị phân cho yêu cầu tra cứu (mã lệnh 1 byte, độ dài 2 byte, khóa UTF-8) bằng DataOutputStream; viết hàm đọc tương ứng và kiểm thử ghi rồi đọc lại.

Tiếp theo

Stream: character stream, charset và framing message